

Fakturama Image-to-Cash Automation

Design Document – Part 1

1. Objective & scope

Build a system that takes a single order image as its only input and produces a saved, verified Order and its linked Invoice inside Fakturama — resolving or creating the Debtor and Product master records as needed, applying the extracted payment status, and verifying every step before moving on. No hardcoded coordinates and no dependence on a fixed UI layout.

Success criteria

- The flow completes as: order image → OCR/LLM → structured order data → automation script → Fakturama UI → saved Order + linked Invoice, with stage verification plus an independent check against the HSQldb file.

Scope tiers (timebox-driven decisions)

Tier	Included
Must (core path)	One clean, printed order image → Order + Invoice, single or few items
Should (if there is time)	LLM-based normalization of messy OCR output

2. System overview & architecture

Three cooperating components:

- Extraction layer — OCR (with optional LLM) turns the image into structured data: order date, external reference, debtor info, item rows (SKU, description, qty, unit net price, VAT %, discount), payment method/status, and totals.
- Grounding / UI layer — Use pywinauto library to find every Fakturama control by accessible name and control type to avoid relying on hardcoded coordinates.
- Main entry point — The workflow is executed in sequential order via a central Python script acting as the primary entry point.

Data flow: order image → OCR/LLM → structured order data → automation script → Fakturama UI → saved Order + linked Invoice, with stage verification plus an independent check against the HSQldb file.

I split the flow into layers, each with its own Python script, mainly so it's easy to test each part on its own. The plan is to start with OCR and add an LLM for normalization if the OCR output needs it; because the layers are separate, I can adjust the extraction without touching the automation. It also keeps the code organized and each step of the flow easy to maintain.

3. Grounding strategy (UIA)

Principles

- Drive Fakturama through Microsoft UI Automation (UIA) using pywinauto.
- Controls are located by their accessible name and control type, not by pixel coordinates. Coordinates are a bad idea here — the moment the window opens in a slightly different position or size, hardcoded clicks land on the wrong element.
- Sometimes the name shown on screen is not the same as the exposed element name, so we use the accessible name. In Fakturama, for example, the toolbar "Order" action is exposed to automation as "Create: New Order".
- Prefer semantic actions over simulated mouse clicks — `invoke()` to press a button, `set_text()` to fill a field — and use the keyboard (`type_keys`) as a fallback, since Fakturama is a Java/SWT (Eclipse RCP) app whose UIA coverage is good but not perfect.
- By "semantic action" I mean telling the element what to do by name, instead of "move the mouse to (400, 100) and click": find the button named "Save" and trigger it.
- Verify after each action: read the control's state back and confirm it has the expected value before moving on.

4. Image-extraction and normalization strategy

Pipeline

1. Preprocess: Clean the input image before handing it to the OCR.
 - a. Load the image file
 - b. Scale/Resize the image to an optimal resolution so the characters are distinct
 - c. Deskewing the image so the text is aligned perfectly for the ocr to be able to read
 - d. Binarize the image to remove any noise leaving only sharp readable text

There are multiple libraries to preprocess the input image. The most common are Pillow and OpenCV. I will mainly use pillow as it is suitable for basic tasks, while OpenCV is for heavier tasks so if needed for optimizing, I will add it.

2. OCR: raw text lines + coordinates.

Engine	Cost	Strength	Weakness
Tesseract	Free	Works on any Python version, No API Key, most configurable.	post-processing required.
Windows built-in OCR	Free	Decent on printed text; zero-install.	Wrapper wheels can lag behind newer Python versions.
LLM vision (GPT-4o / Claude / Gemini)	Paid per use	Best on messy scans. Can output structured JSON directly	Non-deterministic + needs an API Key.

I primarily chose Tesseract because it works on any Python version, needs no API key, and is easy to configure for printed order images. The extraction layer is independent, so if OCR turns out to be too noisy I can switch engines or add an LLM without touching the rest of the flow.

3. Normalize: OCR only gives raw text with positions, so normalization rebuilds the structure: debtor block, item rows, and totals, with numbers parsed in European format and payment names mapped to Fakturama's codes. I'll do this rule-based — because it's a simple and deterministic approach, keeping the extraction layer separate so an LLM can be added later if needed; validation then always checks that rows are complete and totals add up before anything is entered.

4. Validate: OCR and normalization are fuzzy. If they misread something, a wrong order might be created, so I will use Deterministic checks to validate that every item row is correctly filled and the math is consistent. Any mismatch would stop the flow for manual review.

5. The Order Flow

#	Stage	Core Action	Verification Gate
1	Order Initialization	Create a new order, set metadata (Date, Customer Reference, Tax Mode).	Read back Date and Reference fields.
2	Debtor Management	Search and select an existing debtor, or create a new contact profile if not found.	Verify billing and shipping addresses populate in the order.
3	Product Management	Select existing product by SKU (or create a new one with computed gross price), then fill line details.	Confirm line pricing and order totals compute correctly.
4	Order Finalization	Finalize order details, save, and trigger the follow-up Invoice document generation.	Confirm the saved Order row and document link appear under Data.
5	Invoice Completion	Confirm fields copied from the order, set payment details/status, and save.	Verify final Invoice and open Order records.

6. Verification strategy

- UI read-back after every interaction (values, enabled state, open dialogs).
- After each step (Filling an input field), it checks if the input actually matches the extracted data and if it doesn't then the program would stop for a manual review.